

① Responsive web Design

Ans. Responsive web design is an approach to designing websites that automatically adjust their layout, size and content according to the screen size of the device.

Importance:-

- i) works on mobile, tablet and desktop.
- ii) Improves user experience.
- iii) Avoids horizontal scrolling.
- iv) Provides better accessibility.
- v) Reduces need for separate websites.

CSS techniques

- i) Media Queries - Apply different styles for different styles for screen sizes.
- ii) Flexible layouts - uses %, em, rem, vw, and vh instead of fixed sizes.
- iii) Flexbox - Arranges elements in rows or columns.
- iv) CSS Grid - creates responsive rows and columns.
- v) Responsive images - use max-width: 100%.

Example:-

```
<!DOCTYPE html>
<html>
<head>
<style>
  .container {
    display: flex;
    gap: 20px;
  }
  .box {
    flex: 1;
    padding: 20px;
    background: lightblue;
  }
  @media (max-width: 600px) {
    .container {
      flex-direction: column;
    }
  }
</style>
</head>
```

Best practices:- Use readable fonts, touch-friendly buttons, responsive images, proper spacing and test the website on different devices.

② Form Validation and JavaScript Events

Ans client-side validation checks user input in the browser if form is submitted to the server.

Purpose:-

- i) prevent invalid data.
- ii) Give immediate feedback.
- iii) Reduce unnecessary server requests.
- iv) improve user experience.

Javascript event handling

Javascript can use events such as:

- submit
- click
- input
- change
- blur

The submit event can be used to validate the form before sending data to the server.

Example

```
<form id = "myForm">
```

Email:

```
<input type = "text" id = "email">
```

phone:

```
<input type = "text" id = "phone">
```

```
<button type = "submit">submit </button>
```

```
</form>
```

```
<script>
```

```
document.getElementById("myForm").addEventListener("submit", function(e) {
```

```
    let email = document.getElementById("email").value;
```

```
    let phone = document.getElementById("phone").value;
```

```
    let emailPattern = /^[^\s@+@[\s@]+.[^\s@]+$/;
```

```
    let phonePattern = /^[0-9]{10}$/;
```

Role of Regular Expressions

Regular expressions (Regex) define patterns for validating input.

Examples:

- Email → checks name@domain.com format.
- phone → checks whether the input contains exactly 10 digits.

Improvements

- i) Show error messages near the input field.
- ii) Use HTML5 attributes such as `required`, `type="email"` and `pattern`.
- iii) Do server-side validation also, because client-side validation alone is not secure.
- iv) Sanitize user input to prevent attacks such as XSS.

3) CSS Box Model, positioning and Media Queries.
The CSS box model describes how every HTML element is represented as a rectangular box.

Components

- i) content
- ii) padding
- iii) Border
- iv) Margin

CSS positioning

Static: Normal document flow.

Relative: Moves relative to its normal position.

Absolute: positioned relative to a positioned ancestor.

Fixed: Remains fixed relative to the browser window.

Sticky: Behaves relatively until a specified scrolling position.

Causes of layout breaking

- Fixed widths and heights.
- Excessive margins/padding.
- overflowing images.
- Incorrect positioning.
- Poor use of float.
- Lack of responsive CSS.

Media Queries:- Media queries apply CSS based on screen size.

```
.container {  
  width: 80%;  
}  
@media (max-width: 600px) {  
  .container {  
    width: 100%;  
  }  
}
```

Best Practices

- i) use responsive units such as %, rem, vw.
- ii) use flexbox / Grid.
- iii) use box-sizing: border-box.
- iv) Avoid unnecessary fixed widths.
- v) use media queries.
- vi) Make images responsive.

Q) DOM and Dynamic web Content.

Ans: Dom (Document object Model) is a programming representation of an HTML document as a tree of objects/elements.

Example:-

```
HTML  
|  
+-- HEAD  
|  
+-- BODY  
    |  
    +-- H1  
    +-- P  
    +-- BUTTON
```

Role of DOM

The DOM allows Javascript to:

- i) Access HTML elements
- ii) change text and styles.
- iii) Add or remove elements.
- iv) change attributes.
- v) Handle user events.

Javascript DOM manipulation example

```
<h2 id = "title">welcome</h2>
<button onclick = "changeText()">click me</button>
<script>
function changeText() {
document.getElementById("title").textContent = "welcome to
Internet Programming!";
}
</script>
```

when the button is clicked, javascript changes the heading without reloading the complete webpage.

Common DOM methods

```
document.getElementById("id");
document.querySelector(".class");
element.textContent = "New Text";
element.style.color = "red";
element.setAttribute("title", "Hello");
```

Performance and UX improvements

- Update only the required DOM elements.
- Avoid unnecessary DOM manipulation.
- Use event delegation for many elements.
- Use efficient selectors.
- Minimize repeated layout/reflow operations.
- Give immediate visual feedback to users.

Conclusion:- DOM manipulation with Javascript enables dynamic, interactive webpages without requiring a full page reload.